极客时间

读者互动 Bot

从单向推送到双向交互 — 工程思想：风险分层

第 4 周 · 第 15 节

讲师：黄佳

极客时间黄佳老师出品

从推送到互动

极客时间

第 14 节：单向推送

Pipeline → Telegram → 用户
用户被动接收
想找文章？翻聊天记录
想看统计？不支持

信息推送

第 15 节：双向互动

用户 → Bot → 搜索/指令 → 回复
用户主动提问
/search AI Agent
/stats 查看统计
/latest 5 最近文章
知识服务

极客时间

工程思想：风险分层

Not all operations are equal — classify by risk level

不是所有操作都一样危险。按风险等级分类，对不同等级采取不同策略。

极客时间黄佳老师出品

三层风险分类

等级	颜色	操作类型	策略	示例
低风险	绿色	读操作	自动执行	搜索、查询、统计
中风险	黄色	写操作	确认后执行	添加文章、更新标签
高风险	红色	删操作	人工审批	删除文章、清空数据

极客时间

风险分层 — 代码实现

极客时间

knowledge_bot.py | python

```python
RISK_LEVELS = {
    "green": {
        "operations": ["search", "query", "stats", "list"],
        "strategy": "auto_execute",
        "description": "读操作 — 直接执行",
    },
    "yellow": {
        "operations": ["add", "update", "tag", "edit"],
        "strategy": "confirm_first",
        "description": "写操作 — 先确认再执行",
    },
    "red": {
        "operations": ["delete", "clear", "reset", "drop"],
        "strategy": "human_approval",
        "description": "删操作 — 必须人工审批",
    },
}
```

风险分层 — 通用场景

极客时间

场景	绿色（自动）	黄色（确认）	红色（审批）
数据库	SELECT / COUNT	INSERT / UPDATE	DELETE / DROP
Git	status / log / diff	add / commit / push	force-push / reset --hard
API	GET	POST / PUT	DELETE
Claude Code	Read / Grep / Glob	Write / Edit	Bash (rm/curl)

OpenClaw 中的风险分层：allowed-tools

SKILL.md | yaml

```yaml
---
name: knowledge-query
description: 搜索和查询知识库文章
allowed-tools:          # 只开放绿色工具
  - Read                # 读文件
  - Grep                # 搜索内容
  - Glob                # 查找文件
  # - Write             # 黄色 — 不开放
  # - Edit              # 黄色 — 不开放
  # - Bash              # 红色 — 绝不开放
---
```

极客时间

Bot 架构设计

意图识别 → 搜索 / 指令 → 回复

极客时间

极客时间

Bot 架构图

用户消息
意图识别
detect_intent
← 规则匹配，不用 LLM
Command
指令处理
/search
/latest
/stats
/help
Search
知识检索
search_articles()
加权匹配
格式化回复 → 用户

极客时间

意图识别 — 规则匹配流程

极客时间

输入文本
|
▼
以 / 开头? ——Yes——→ return 'command'
|
No
|
▼
包含搜索关键词? ——Yes——→ return 'search'
(查/搜/找/关于)
|
No
|
▼
return 'search' ← 默认兜底

为什么默认是 search?

- 搜索是绿色操作（零风险）
- 搜不到最多返回空结果
- 不会产生副作用

极客时间

极客时间黄佳老师出品

意图识别 — 代码实现

knowledge_bot.py | python

```python
def detect_intent(text: str) -> str:
    """识别用户意图（规则匹配，不用 LLM）"""
    text = text.strip()

    # 规则 1：斜杠指令
    if text.startswith("/"):
        return "command"

    # 规则 2：搜索关键词
    search_keywords = ["查", "搜", "找", "搜索", "查找",
                       "有没有", "哪些", "关于"]
    for kw in search_keywords:
        if kw in text:
            return "search"

    # 规则 3：默认搜索（绿色兜底）
    return "search"
```

知识检索 — 加权匹配算法

极客时间

knowledge_bot.py | python

```python
def search_articles(query, articles, top_k=5):
    """搜索文章 — 加权匹配"""
    results = []
    keywords = query.lower().split()

    for article in articles:
        score = 0
        title = article.get('title', '').lower()
        tags = [t.lower() for t in article.get('tags', [])]
        summary = article.get('summary', '').lower()

        for kw in keywords:
            if kw in title:    score += 10  # 标题最重要
            for tag in tags:
                if kw in tag:  score += 5   # 标签次之
            if kw in summary:  score += 3   # 摘要最低

        if score > 0:
            results.append({'article': article, 'score': score})

    results.sort(key=lambda x: x['score'], reverse=True)
    return results[:top_k]
```

搜索权重设计

匹配位置	权重	理由	示例
标题	+10	标题 = 文章主题，最精准	搜 GPT→ 标题含 GPT 的文章
标签	+5	标签 = 作者分类，较精准	搜 LLM → 标签含 LLM 的文章
摘要	+3	摘要可能提及但非主题	搜“训练”→ 摘要提到训练的文章

极客时间黄佳老师出品

指令系统 — 四个斜杠指令

极客时间

指令	功能	风险等级	示例
/search <关键词>	搜索文章	绿色	/search AI Agent
/latest [N]	最近 N 篇	绿色	/latest 5
/stats	知识库统计	绿色	/stats
/help	帮助信息	绿色	/help

指令处理 — handle_command()

knowledge_bot.py | python

```python
def handle_command(text, articles):
    """处理斜杠指令"""
    parts = text.strip().split(maxsplit=1)
    cmd = parts[0].lower()
    args = parts[1] if len(parts) > 1 else ""

    if cmd == "/search":
        if not args:
            return "用法：/search <关键词>"
        results = search_articles(args, articles)
        return format_results(results)
```

knowledge_bot.py | python

```python
    elif cmd == "/latest":
        n = int(args) if args.isdigit() else 5
        n = min(n, 20)  # 最多 20 条
        sorted_a = sorted(articles,
            key=lambda a: a.get('date', ''), reverse=True)
        return format_latest(sorted_a[:n])

    elif cmd == "/stats":
        return format_stats(articles)

    elif cmd == "/help":
        return HELP_TEXT

    return f"未知指令：{cmd}\n输入 /help 查看帮助"
```

OpenClaw Skill 集成 — knowledge-query

skills/knowledge-query/SKILL.md | yaml

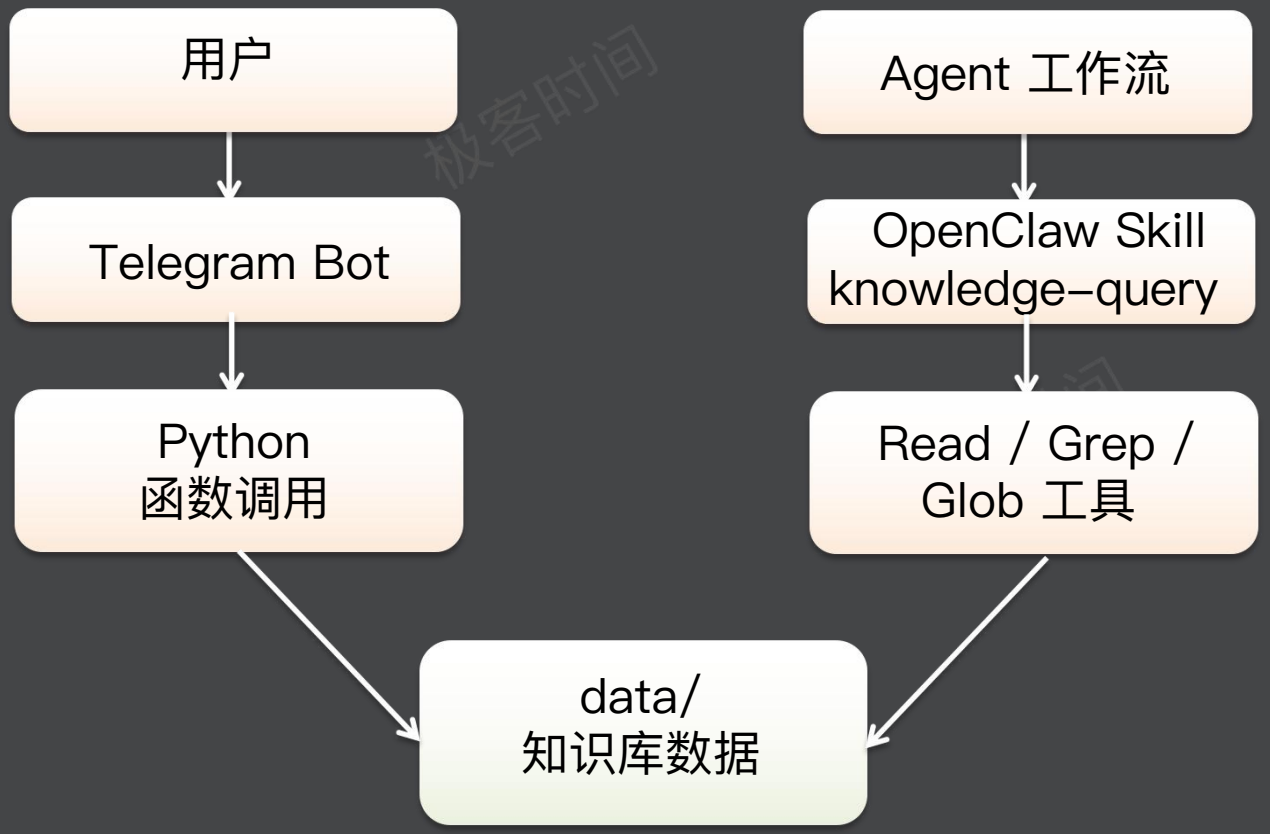
```yaml
---
name: knowledge-query
description: 当用户需要搜索、查询或统计知识库文章时使用
allowed-tools:
  - Read
  - Grep
  - Glob
---

# 知识库查询 Skill

## 能力范围
- 搜索文章（按关键词、标签）
- 查看最近文章
- 统计知识库概况

## 操作约束
- 只读操作，不修改任何文件
- 搜索范围限定在 data/ 目录
- 返回 Top 5 结果
```

两条路径，同一个知识库



极客时间黄佳老师出品

极客时间

风险分层三原则

极客时间

1. 默认最小权限
2. 读写分离
3. 不可逆加权

未分类操作 → 默认高风险
读操作和写操作 → 永远不在同一权限等级
操作是否可逆 → 分级的核心依据

本节作业

极客时间

任务 1：实操 1：实现 knowledge_bot.py（20 分钟）

Python 模块:search_articles 加权匹配(标题+10/标签+5/摘要+3) + detect_intent 意图识别 + handle_command 处理 /search /latest /stats /help。用测试数据验证。

任务 2：实操 2：包装成 OpenClaw Skill（15 分钟）

创建 ~/ai-knowledge-base/v4-production/openclaw/skills/knowledge-query/SKILL.md (13-2 已把 workspace 切到 v4)。Skill 内只用 read 工具(从 index.json 入口),让 Bot 通过对话触发搜索。Telegram 发"搜一下 agent"得到 top 5 结果。

任务 3：扩展（可选 · 跑通主线再玩）

加 daily-digest Skill(每日简报) / LLM 重排 / 同义词扩展 / 分页 /next 等等。不强求,挑感兴趣的玩。

极客时间

本节小结

极客时间

工程思想：风险分层

读（绿色自动）/ 写（黄色确认）/ 删（红色审批），不可逆操作必须严格

知识检索

加权匹配（标题 ×10 / 标签 ×5 / 摘要 ×3），规则引擎比 LLM 更稳定更快

意图识别

正则规则（不调 LLM），延迟 <10ms，确定性输出——指令系统首选

Bot 指令体系

/search · /latest · /stats · /help，每个指令都有异常输入兜底

OpenClaw Skill 集成

knowledge-query SKILL.md 让 Telegram 用户直接查询知识库

下一节：第 16 节 部署上线与运维 — 完结篇

极客时间黄佳老师出品